SIMATS ENGINEERING

CO2_AT3 – ERROR ANALYSIS TASK

Course Name: Software Testing

Course Code: CSA37

Name: Charuthya P

Registration number: 192421148

QUESTION 1

A tester designs test cases for an online shopping system using equivalence partitioning but incorrectly groups input ranges (e.g., valid and invalid inputs mixed in one class). Additionally, boundary values are not considered. Analyze the given test design and identify all errors in equivalence class partitioning and boundary value analysis. Examine the root causes of these mistakes with proper reasoning. Propose corrected test cases with proper classification and boundary coverage. Justify how your corrections improve defect detection effectiveness.

ANSWER

Errors identified

- Valid and invalid values are grouped into the same equivalence class, even though they may produce different system behaviour.
- The classes do not cover all possible inputs or may overlap, so some inputs are untested or ambiguously classified.
- Boundary Value Analysis (BVA) is omitted; defects commonly occur at minimum and maximum limits and just inside or outside them.

Root causes

- The tester has not defined the input rules and constraints before partitioning.
- The tester assumes representative values are sufficient and overlooks off-by-one errors, range checks, and invalid data handling.

Corrected example: assume the shopping quantity field accepts integers from 1 to 10.

Equivalence classes: valid integers 1–10; invalid integers below 1; invalid integers above 10; invalid non-integer/text input.

BVA test cases: -1, 0, 1, 2, 9, 10, 11 (plus text such as “abc” and a decimal such as 1.5 if the field requires integers). Expected results: 1–10 are accepted; all out-of-range or wrong-type inputs are rejected with a clear validation message.

Justification: separate classes represent distinct behaviours, while boundary tests target off-by-one and limit-checking defects. Together they improve input coverage without requiring every possible value to be tested.

QUESTION 2

A QA engineer performs random testing on a banking application and claims high coverage without using any structured techniques. Critical defects in transaction limits and boundary conditions are missed. Evaluate the testing approach and identify errors in strategy and coverage assumptions. Analyze the root causes behind missing critical defects. Suggest appropriate black box testing techniques to correct the approach. Justify how these techniques ensure better test adequacy and reliability.

ANSWER

Evaluation and errors

- Random testing may generate many inputs but does not guarantee coverage of important requirements, partitions, boundaries, or business rules.
- The claim of high coverage is unsupported unless coverage is measured against requirements, input classes, and risk areas.
- Rare but important cases, such as exact transaction limits and invalid amounts, may not be randomly selected.

Root causes

- No risk-based test plan or traceability from banking requirements to test cases.
- No deliberate selection of minimum, maximum, just-below, and just-above transaction values.
- No systematic combinations of account type, transaction type, balance, and limit rules.

Corrected black-box techniques

- Equivalence Class Partitioning: valid amounts, zero/negative amounts, values over the limit, malformed input, and insufficient-balance cases.
- Boundary Value Analysis: for a permitted range of ₹1–₹50,000, test ₹0, ₹1, ₹2, ₹49,999, ₹50,000, and ₹50,001, adapting the limits to actual requirements.
- Decision Table Testing: cover combinations such as sufficient/insufficient balance, limit exceeded/not exceeded, and authorised/unauthorised transaction.
- State Transition Testing: verify states such as initiated, authorised, processed, failed, and reversed, including invalid transitions and retries.

Justification: structured techniques make the test set traceable and repeatable, target high-risk conditions, and expose boundary and rule-interaction defects. Random tests can supplement, but not replace, these techniques.

QUESTION 3

A developer performs white box testing but creates an incomplete control flow graph, missing decision nodes and loops, resulting in poor path coverage. Analyze the errors in the control flow graph and path testing approach. Identify the root causes for inadequate test coverage. Propose corrections by constructing a proper control flow representation and identifying all independent paths. Justify how improved path testing ensures better coverage and defect detection.

ANSWER

Errors and causes

- A control flow graph (CFG) must represent each statement/basic block as a node and each possible transfer of control as a directed edge.
- Omitting decision nodes hides true/false branches; omitting loop edges hides zero, one, and repeated iteration behaviour.

- An incomplete CFG produces an incorrect view of reachable paths and may leave branches and loop exit conditions untested.
- Likely causes include drawing the graph from an incomplete understanding of the code, not reviewing each condition, and not using coverage criteria.

Corrected example CFG (generic code)

1 Start → 2 Read input → 3 Is input valid?

3 (True) → 4 Initialise counter → 5 Is counter < n?

3 (False) → 9 Display error → 10 End

5 (True) → 6 Process item → 7 Increment counter → 5 (loop back)

5 (False) → 8 Display result → 10 End

Independent paths for this example

- P1: 1–2–3(False)–9–10: invalid input and error exit.
- P2: 1–2–3(True)–4–5(False)–8–10: valid input with n = 0, so the loop executes zero times.
- P3: 1–2–3(True)–4–5(True)–6–7–5(False)–8–10: valid input with one loop iteration.

For a loop that can execute many times, additional iteration tests should be selected according to the loop requirements; there are infinitely many possible paths if unrestricted looping is included. The listed paths are a basis for the generic example, not every possible execution sequence.

Justification: a complete CFG makes branches and loop-back edges visible. Basis-path testing exercises independent control-flow paths, while branch and loop tests expose incorrect conditions, skipped processing, faulty termination, and off-by-one errors.

QUESTION 4

A testing team reports 100% statement coverage for a module but fails to detect logical errors in conditional statements. Evaluate the adequacy of the testing performed and identify gaps in coverage criteria. Analyze why statement coverage alone was insufficient. Recommend improved white box testing strategies such as branch or path coverage. Justify how the revised approach enhances overall test effectiveness and error detection.

ANSWER

Evaluation

100% statement coverage means every executable statement ran at least once. It does not prove that every decision outcome or relevant combination of conditions was tested.

Why gaps remain

- A test may execute the body of an if statement but never test the false outcome.
- Compound conditions may contain defects even when the overall decision happens to evaluate as expected.
- Different paths may set variables or update data differently, creating errors that statement coverage cannot reveal.

Recommended strategies

- Branch coverage: execute both true and false outcomes of each decision. For example, for if (balance >= amount), test a sufficient balance and an insufficient balance.
- Condition coverage: make each atomic condition evaluate to both true and false at least once.
- Modified Condition/Decision Coverage (MC/DC), where appropriate: demonstrate that each condition can independently affect the decision outcome.
- Basis-path testing: use the CFG and cyclomatic complexity to select a set of independent paths.

Illustration: for if (A && B), tests (A=true, B=true) and (A=false, B=true) cover both decision outcomes, but do not make B false. Add (A=true, B=false) to exercise B as false. The exact test set should match the chosen coverage criterion.

Justification: branch and condition-oriented criteria exercise outcomes and logical combinations that statement coverage can miss, improving detection of incorrect predicates, missing else behaviour, and faulty control flow.

QUESTION 5

A tester uses Equivalence Class Partitioning for an age-validation field (0-100) but defines only two classes: Negative numbers and Positive numbers. Analyze the errors in this partitioning strategy. Identify why this fails to account for valid vs. invalid age limits and the absence of Boundary Value Analysis. Propose a corrected set of partitions and justify how they ensure more robust input validation.

ANSWER

Errors

- “Positive numbers” combines valid ages (1–100) with invalid ages above 100.
- Age 0 is valid under the stated range but is neither negative nor positive, so the classes do not cover all inputs.
- The classes do not distinguish non-numeric input or non-integer ages if only whole-number ages are allowed.
- No boundary tests are specified for 0 and 100.

Corrected equivalence classes (assuming whole-number ages)

- Invalid: age < 0.
- Valid: age from 0 through 100 inclusive.
- Invalid: age > 100.
- Invalid format/type: letters, symbols, blank input, and decimals if only integers are accepted.

Boundary Value Analysis test cases: -1, 0, 1, 99, 100, 101. Also test blank input and a non-numeric value such as “abc”. Expected results: 0–100 are accepted; negative and above-100 values are rejected; invalid formats are handled safely.

Justification: the partitions are mutually exclusive and cover meaningful input behaviours. Boundary tests detect inclusive/exclusive comparison mistakes and off-by-one errors.

QUESTION 6

A developer claims full Condition Coverage for a complex if (A && B) expression but provides no test cases or evidence. Analyse the claim and explain how condition coverage should be demonstrated.

ANSWER

The supplied assignment text truncates Question 6 after "if (A && B"; therefore, the answer below addresses the visible condition-coverage issue.

A claim of full condition coverage cannot be accepted without a test set and results showing that every atomic condition has evaluated to both true and false at least once. For A && B, one suitable set is:

- Test 1: A=true, B=true → decision true.
- Test 2: A=false, B=true → decision false; A is false.
- Test 3: A=true, B=false → decision false; B is false.

Across these tests, A is true and false, and B is true and false. The tests also show both decision outcomes. In short-circuit languages, B may not be evaluated when A is false, so the third test is important for demonstrating B=false. The test cases should be recorded with actual execution evidence.

QUESTION 7

A QA team applies Decision Table Testing for a discount system but misses several combinations of "Premium Member" and "Coupon Applied," leading to price calculation errors. Analyze the errors in the construction of the decision table. Identify why the current approach is incomplete. Propose a revised, exhaustive decision table and justify its effectiveness in preventing production defects.

ANSWER

Errors and root causes

- Some combinations of the two binary conditions were omitted.
- The team may have focused on typical scenarios instead of systematically enumerating all condition combinations.
- Actions and expected discounts may not have been explicitly assigned to each rule.

Assumption for the corrected table: premium members receive a 10% discount, a valid coupon gives 5%, and both discounts stack additively. Actual business requirements must replace this assumption if different.

Rule	Premium Member?	Coupon Applied?	Expected action	Total discount
R1	No	No	Apply no discount	0%
R2	No	Yes	Apply coupon discount	5%
R3	Yes	No	Apply member discount	10%
R4	Yes	Yes	Apply both discounts	15%

Justification: two independent yes/no conditions create $2^2 = 4$ combinations. Testing all four rules prevents omitted combinations and verifies the correct action for each. Add tests for invalid/expired coupons and discount caps if required by the actual specification.

QUESTION 8

During unit testing of a looping structure, a tester checks only the first and last iterations (n=1 and n=max). Defects occurring at n=0 and n=max+1 are missed. Evaluate the errors in this Loop Testing strategy. Analyze the root causes behind these omissions. Recommend a corrected approach for testing simple and nested loops, and justify how it ensures path integrity.

ANSWER

Errors

- Testing n=1 and n=max does not test the zero-iteration case or the invalid case just beyond the maximum.
- The strategy overlooks loop entry, exit, termination, and boundary conditions.
- Nested loops introduce interactions between inner and outer loop counts that are not covered by testing only one iteration count.

Corrected approach for a simple loop

- Test 0 iterations, 1 iteration, 2 iterations, a typical number of iterations, max−1, max, and max+1 where the interface permits the value. If max+1 is invalid, verify safe rejection rather than executing an unsupported loop count.
- Check correct initialisation, loop condition, increment/decrement, termination, and output after exit.
- For a nested loop, first test the outer loop at its minimum and typical counts while controlling the inner loop; then test the inner loop at 0, 1, 2, typical, and boundary counts. Include combinations such as outer=0, outer=1/inner=0, outer=1/inner=1, and representative maximum counts.
- For concatenated loops, test each loop independently at its boundaries and then test their interaction.

Root causes and justification

The omissions usually arise from weak boundary analysis and an assumption that first/last valid iterations represent all behaviour. A systematic loop test set exposes off-by-one errors, skipped or repeated iterations, incorrect termination, and failures at empty or over-limit input.

QUESTION 9

An engineer utilizes Error Guessing as the primary testing technique for a safety-critical braking system, ignoring structured black box methods. Identify the errors in relying solely on ad-hoc techniques for high-risk modules. Analyze the impact on test adequacy. Suggest a combination of BVA and State Transition Testing to fix the strategy, justifying the increased safety assurance.

ANSWER

Errors and risks

- Error Guessing depends heavily on individual experience and is difficult to reproduce or prove complete.
- It may omit required input boundaries, operating states, and transitions; there is no systematic evidence that safety requirements are covered.
- For a braking system, missed faults can cause severe harm, so ad-hoc testing alone is inadequate.

Corrected strategy

- Boundary Value Analysis: derive limits from the safety specification, such as minimum/maximum sensor values, pressure, speed, response time, and permitted command ranges. Test just below, at, and just above each boundary, with expected safe behaviour defined by the specification.
- State Transition Testing: model states such as standby, braking requested, braking active, fault detected, and safe fallback. Test valid transitions and invalid transitions, sensor failures, loss of signal, recovery, and emergency-stop behaviour.
- Use requirement traceability, independent review, repeatable test records, and additional risk-appropriate verification. Error guessing can supplement, but not replace, structured tests.

Justification: BVA targets limit-related faults, while state-transition tests verify temporal and mode-dependent behaviour. Together with safety requirements and documented evidence, they provide more systematic assurance; they do not by themselves prove the system is safe.

QUESTION 10

A tester designs a Control Flow Graph (CFG) for a module but fails to represent the “else” branches of multiple nested if statements, leading to 100% statement coverage but 0% branch coverage on error paths. Analyze the errors in graph construction. Identify the root causes for the missing nodes. Propose a corrected CFG and independent paths to ensure branch adequacy.

ANSWER

Errors in the CFG

- Missing else/error edges mean the graph omits feasible control transfers and does not represent the program accurately.
- Nested decisions require separate true and false outgoing edges for each decision; missing edges hide combinations of outcomes.
- 100% statement coverage can be achieved while error branches remain unexecuted; it is not evidence of branch adequacy.

Root causes

- The graph may have been drawn from the intended “happy path” rather than the complete source code.
- The tester may have treated compound/nested conditions as one step and failed to review every false outcome and early exit.

Corrected generic CFG (illustrative code: if A, then if B do success; otherwise error B; otherwise error A)

1 Start → 2 Evaluate A?

2 (True) → 3 Evaluate B?

2 (False) → 6 Error A → 7 End

3 (True) → 4 Success → 7 End

3 (False) → 5 Error B → 7 End

Independent paths

- P1: 1–2(False)–6–7: outer condition A fails.
- P2: 1–2(True)–3(False)–5–7: A succeeds but nested condition B fails.
- P3: 1–2(True)–3(True)–4–7: both conditions succeed.

Test data should force A=false, A=true/B=false, and A=true/B=true. If the actual code has more nested decisions, add paths for every feasible true/false outcome and relevant combinations. Branch coverage requires every branch edge to be executed at least once.

Justification: the corrected graph includes both outcomes for each decision, making error exits visible and testable. Independent path tests expose missing error handling and incorrect nested logic that a happy-path-only test set can miss.